

Dynamic Bonding Curve

0.2.0

Smart Contract Security Assessment

June 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie

Ronny Xing





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	6
4	Key Findings and Recommendations	7
4.1	Incorrect System Program Validation In Anchor 1.0.0	7
4.2	External Transfer Hook Program Risk In TransferHookPool	8
4.3	Potential Self Referral Compatibility Break In Swap Referral Account	9
4.4	Zero Amount Base Transfers Can Be Blocked By Base Transfer Hooks	10
4.5	Informational and Undetermined Issues	12
5	Disclaimer	14



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers, operating systems, IoT devices, and hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple, Google, and Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *Dynamic Bonding Curve* smart contracts, starting on May 11th, 2026, and concluding on June 1st, 2026.

Project Overview

Meteora Dynamic Bonding Curve Release 0.2.0 upgrades the program to Anchor 1.0.2 and introduces Token 2022 transfer hook launch pools with separate creation, swap, fee claim, completion, and event flows. The release also adds a protocol fee claim path based on protocol fee authority, tightens mint authority handling for standard pools, deprecates older protocol fee entry points, and updates migration and fee handling to preserve compatibility between hook and non hook pools.

Audit Scope

The assessment scope contains mainly the smart contracts of the dynamic-bonding-curve program for the *Dynamic Bonding Curve* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Dynamic Bonding Curve
 - Codebase: <https://github.com/MeteoraAg/dynamic-bonding-curve>
 - PR-188
 - Codebase Link: [PR-188](#)
 - Commit Hash: 8b3449d1b8377271fb8b33a0bb8335121e655c92
 - PR-193
 - Codebase Link: [PR-193](#)
 - Commit Hash: 5c3ad521c8847984bc1ed739c37e1f6a50be2610

The issues described in this report have been resolved by the following commit:

- Dynamic Bonding Curve
 - Codebase: <https://github.com/MeteoraAg/dynamic-bonding-curve>
 - Commit Hash: 15e98f293059d0b14f2e8bf86a334c3454b602c2

We listed the files we have audited below:

- Dynamic Bonding Curve Release 0.2.0
 - programs/dynamic-bonding-curve/src/access_control.rs
 - programs/dynamic-bonding-curve/src/const_pda.rs
 - programs/dynamic-bonding-curve/src/constants.rs
 - programs/dynamic-bonding-curve/src/error.rs
 - programs/dynamic-bonding-curve/src/event.rs
 - programs/dynamic-bonding-curve/src/instructions/admin/auth.rs



- programs/dynamic-bonding-curve/src/instructions/admin/ix_close_claim_protocol_fee_operator.rs
- programs/dynamic-bonding-curve/src/instructions/creator/ix_claim_creator_trading_fee.rs
- programs/dynamic-bonding-curve/src/instructions/creator/ix_create_virtual_pool_metadata.rs
- programs/dynamic-bonding-curve/src/instructions/creator/ix_transfer_pool_creator.rs
- programs/dynamic-bonding-curve/src/instructions/creator/ix_withdraw_creator_surplus.rs
- programs/dynamic-bonding-curve/src/instructions/initialize_pool/ix_initialize_virtual_pool_with_spl_token.rs
- programs/dynamic-bonding-curve/src/instructions/initialize_pool/ix_initialize_virtual_pool_with_token2022_transfer_hook.rs
- programs/dynamic-bonding-curve/src/instructions/initialize_pool/ix_initialize_virtual_pool_with_token2022.rs
- programs/dynamic-bonding-curve/src/instructions/initialize_pool/mod.rs
- programs/dynamic-bonding-curve/src/instructions/initialize_pool/process_initialize_virtual_pool_with_token2022.rs
- programs/dynamic-bonding-curve/src/instructions/migration/create_locker.rs
- programs/dynamic-bonding-curve/src/instructions/migration/dynamic_amm_v2/migrate_damm_v2_initialize_pool.rs
- programs/dynamic-bonding-curve/src/instructions/migration/ix_withdraw_migration_fee.rs
- programs/dynamic-bonding-curve/src/instructions/migration/meteora_damm/meteora_damm_claim_lp_token.rs
- programs/dynamic-bonding-curve/src/instructions/migration/meteora_damm/meteora_damm_lock_lp_token.rs
- programs/dynamic-bonding-curve/src/instructions/migration/meteora_damm/migrate_meteora_damm_initialize_pool.rs
- programs/dynamic-bonding-curve/src/instructions/migration/meteora_damm/migration_meteora_damm_create_metadata.rs
- programs/dynamic-bonding-curve/src/instructions/migration/withdraw_leftover.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_claim_protocol_fee.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_claim_protocol_fee2.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_claim_protocol_pool_creation_fee.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_zap_protocol_fee.rs
- programs/dynamic-bonding-curve/src/instructions/operator/mod.rs
- programs/dynamic-bonding-curve/src/instructions/partner/create_config/ix_create_config.rs



- programs/dynamic-bonding-curve/src/instructions/partner/create_config/ix_create_transfer_hook_config.rs
- programs/dynamic-bonding-curve/src/instructions/partner/create_config/mod.rs
- programs/dynamic-bonding-curve/src/instructions/partner/create_config/process_create_config.rs
- programs/dynamic-bonding-curve/src/instructions/partner/ix_claim_partner_pool_creation_fee.rs
- programs/dynamic-bonding-curve/src/instructions/partner/ix_claim_partner_trading_fee.rs
- programs/dynamic-bonding-curve/src/instructions/partner/ix_create_partner_metadata.rs
- programs/dynamic-bonding-curve/src/instructions/partner/ix_withdraw_partner_surplus.rs
- programs/dynamic-bonding-curve/src/instructions/partner/mod.rs
- programs/dynamic-bonding-curve/src/instructions/swap/ix_swap.rs
- programs/dynamic-bonding-curve/src/instructions/swap/ix_swap2_with_transfer_hook.rs
- programs/dynamic-bonding-curve/src/instructions/swap/mod.rs
- programs/dynamic-bonding-curve/src/instructions/swap/process_swap.rs
- programs/dynamic-bonding-curve/src/lib.rs
- programs/dynamic-bonding-curve/src/math/safe_math.rs
- programs/dynamic-bonding-curve/src/params/swap.rs
- programs/dynamic-bonding-curve/src/state/config.rs
- programs/dynamic-bonding-curve/src/state/fee.rs
- programs/dynamic-bonding-curve/src/state/virtual_pool.rs
- programs/dynamic-bonding-curve/src/utils/activation_handler.rs
- programs/dynamic-bonding-curve/src/utils/config_account_loader.rs
- programs/dynamic-bonding-curve/src/utils/mod.rs
- programs/dynamic-bonding-curve/src/utils/pool_account_loader.rs
- programs/dynamic-bonding-curve/src/utils/remaining_accounts.rs
- programs/dynamic-bonding-curve/src/utils/token.rs

Findings

The security audit revealed:

- 0 critical issue
- 1 high issues
- 1 medium issues
- 2 low issues
- 1 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Incorrect System Program Validation In Anchor 1.0.0	High	Fixed
02	External Transfer Hook Program Risk In TransferHookPool	Medium	Acknowledged
03	Potential Self Referral Compatibility Break In Swap Referral Account	Low	Fixed
04	Zero Amount Base Transfers Can Be Blocked By Base Transfer Hooks	Low	Fixed
05	Transfer Hook Signer De Escalation Relies On A Transitive Dependency Version	Informational	Fixed



4 Key Findings and Recommendations

4.1 Incorrect System Program Validation In Anchor 1.0.0

Severity: High

Status: Fixed

Target: Smart Contract

Category: Framework

Description

The program is upgraded to Anchor 1.0.0. This Anchor version has a known validation issue for typed program accounts such as `Program<'info, System>`.

In Anchor `1.0.0~1.0.1`, `Program<T>` uses the program id returned by `T::id()` to decide whether the account should be treated as a typed program or as a generic executable program. Since the System Program id is the default public key, `Program<'info, System>` may be misclassified as a generic program account. As a result, Anchor may only check that the provided account is `executable`, without enforcing that its address is the real System Program address.

```
152 // Special handling for unit type () - only check executable, not
    // program ID
153 let is_unit_type = T::id() == Pubkey::default();
154
155 if !is_unit_type && info.key != &T::id() {
156     return
        Err(Error::from(ErrorCode::InvalidProgramId).with_pubkeys(
            (*info.key, T::id())));
157 }
158 if !info.executable {
159     return Err(ErrorCode::InvalidProgramExecutable.into());
160 }
```

[lang/src/accounts/program.rs#L152-L160](#)

The affected code pattern is:

```
pub system_program: Program<'info, System>,
```

The program contains multiple instructions using this pattern for account initialization, account resizing, rent funding, and other system program operations.

Impact

An attacker may be able to pass a non System executable program where the real System Program is expected. This can affect instructions that depend on System Program validation for account creation, reallocations, rent transfers, or PDA based setup flows. The



practical impact depends on each call path, but the affected validation is security critical and applies broadly across the program.

Recommendation

It's recommended to upgrade Anchor to version 1.0.2, where this issue is fixed in this commit [commit/3eb1fb0427d4fed0e944b627aef76db651a93aab](https://github.com/project-serapion/anchor/commit/3eb1fb0427d4fed0e944b627aef76db651a93aab).

If the program continues to use Anchor 1.0.0 or 1.0.1, explicitly constrain every System Program account to the canonical System Program address, for example:

```
#[account(address = anchor_lang::system_program::ID)]  
pub system_program: Program<'info', System>,
```

Mitigation Review Log

Fixed in the commit 99c5c16061c30a3f2bc57f08c22cc614846c7dc8.

4.2 External Transfer Hook Program Risk In TransferHookPool

Severity: Medium

Status: Acknowledged

Target: Smart Contract

Category: Design

Description

TransferHookPool configures the base mint with an external transfer hook program selected through the transfer hook config. DBC validates that the configured program is executable and rejects DBC, SPL Token, and Token 2022 as hook programs, but it does not control the hook program's code, upgrade authority, or future transfer policy after the pool is created.

1. If the hook program is upgradeable, the code that users rely on during pool creation may not be the same code that runs later. A hook program can be upgraded after liquidity and users are already in the pool, including through a compromised upgrade authority or an unsafe project-side upgrade.
2. Even without a malicious upgrade, the hook program can fail because of its own implementation. Bugs, incorrect account validation, unavailable extra accounts, bad configuration, dependency changes, or excessive compute usage can make valid base token transfers fail.
3. DBC's normal transfer hook handling assumes the external hook remains usable until the pool reaches the lifecycle point where the hook is removed. The reviewed code does not provide a separate pre-completion recovery path that lets DBC bypass, replace, or remove a broken hook when the external program itself is the reason base transfers fail.



Impact

If the external hook rejects or fails during transfer execution, DBC base token transfers can be blocked. This can prevent users from buying or selling through the transfer hook pool, block base-side referral transfers, and block partner or creator claims that require transferring base tokens.

The pool can also become stuck before migration if curve completion requires a base transfer that the hook rejects, because the normal hook removal path is reached only after curve completion. If users cannot exit, refunds are hard to calculate on-chain because the pool only stores aggregate reserves and fees. Swap events can help off-chain analysis, but the program does not keep per-user cost records for an automatic refund.

Recommendation

It is recommended to define an explicit trust and recovery model for transfer hook pools. This can include:

1. Requiring the hook program to be immutable at configuration creation, for example by checking that its ProgramData upgrade authority is `None`.
2. Adding an operator gated emergency hook removal path so base token transfers can proceed without the hook.
3. Restricting supported hook programs through an approved program list.

If emergency hook removal is supported, it is recommended to also define the recovery outcome before enabling this feature in production. The available recovery models include:

1. Simply removing the hook and allowing the pool to continue operating.
2. Adding an on chain refund rule based on a predefined formula or snapshot.
3. Handling refunds off chain using swap events and project records.

Mitigation Review Log

Meteora Team: This is an inherent risk for Transfer Hook program. We'll keep it loose for now since it's meant to be an experiment. If we see adoption and have use case for it, we can add restrictions and recovery methods later.

4.3 Potential Self Referral Compatibility Break In Swap Referral Account

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Framework

Description

Anchor 1.0 introduces `duplicate mutable account checks` for mutable account types that serialize or write back on exit, including mutable InterfaceAccount token accounts.



In the `swap` / `swap2` / `swap2_with_transfer_hook` instructions, the `referral_token_account` token account and the user input token account are both `mutable` token accounts. In Anchor 0.x, the `referral_token_account` account could be the same as the user base / quote token account, allowing the referral fee to be sent back to the user's token account.

However, After the Anchor 1.0 upgrade, this duplicate mutable account combination will be rejected.

Impact

If any users or integrators rely on using the base/quote token account as the referral fee account, those swaps will fail after the upgrade.

Recommendation

If self referral is supported in `swap` / `swap2` / `swap2_with_transfer_hook` instructions, it's recommended to mark the `referral_token_account` account with `dup` .

Mitigation Review Log

Fixed in the commit 3114aeb355087abeb217a1e64a347e6f5466b616.

4.4 Zero Amount Base Transfers Can Be Blocked By Base Transfer Hooks

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In `claim_creator_trading_fee2` and `claim_trading_fee2` , the claimed base and quote amounts are calculated independently from the stored fee balances. This means the base amount can be zero while the quote amount is nonzero.

However, the claim handlers execute the base token transfer before the quote token transfer. If `token_base_amount` is zero, the transfer still routes through the base mint transfer hook path for transfer hook pools.



```
107     transfer_token_from_pool_authority(  
108         ...  
109         token_base_amount,  
110         parsed_transfer_hook_accounts.transfer_hook_base,  
111     )?;  
112  
113     transfer_token_from_pool_authority(  
114         ...  
115         token_quote_amount,  
116         None,  
117     )?;
```

[programs/dynamic-bonding-curve/src/instructions/partner/ix_claim_partner-trading_fee.rs#L107-L125](#)

The same pattern also exists in the swap referral fee path. In `process_swap`, when a referral token account is provided and fees are collected on the base token, the program calls the base referral transfer unconditionally.

```
295     if let Some(referral_token_account) = referral_token_account.as_ref() {  
296         if fee_mode.fees_on_base_token {  
297             transfer_token_from_pool_authority(  
298                 ...  
299                 swap_result.referral_fee,  
300                 transfer_hook_base_referral_accounts,  
301             )?;  
302         }  
303     }
```

[programs/dynamic-bonding-curve/src/instructions/swap/process_swap.rs#L295-L308](#)

The referral fee is calculated with rounding down, so it can be zero for small swaps. Even in that case, the transfer helper does not skip the transfer.

When the base mint has a transfer hook, the helper still routes the zero amount transfer through the transfer hook path, regardless of whether any token is actually moved. This can invoke the external transfer hook program with amount equal to zero. A restrictive hook program may reject zero amount transfers, require additional state, or otherwise fail. As a result, an unrelated zero amount base transfer can block the later quote fee transfer or the swap itself.

Impact

A failing or restrictive base transfer hook can block actions that are not actually transferring any base token.

For fee claims, this can prevent partner or creator quote fee claims when no base fee is being claimed. The transaction reverts before the quote token transfer is executed.

For swaps, this can cause valid swaps with a referral account to fail when the computed



base referral fee is zero. The swap may still require base referral hook accounts, or may be blocked by the hook program, even though no referral base token is being transferred.

Recommendation

It is recommended to skip each token transfer when the corresponding amount is zero.

Partner and creator quote fee claims should not invoke the base token transfer or its transfer hook when `token_base_amount` is zero. Similarly, the swap referral fee path should not invoke the base referral transfer or its transfer hook when `swap_result.referral_fee` is zero.

Mitigation Review Log

Fixed in the commit `3e26a8efe0e875a2ef7f19c9ad5c3e6d3cc942a4` and `97d8eabb6e185f9ce720c4fe864bd0840e6ed1f5`.

4.5 Informational and Undetermined Issues

Transfer Hook Signer De Escalation Relies On A Transitive Dependency Version

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

DBC uses `spl-transfer-hook-interface 2.1.0` for Token-2022 transfer hook extra accounts, and the reviewed Cargo.lock currently resolves its transitive `spl-tlv-account-resolution` dependency to `0.11.1`.

However, the manifest constraint from `spl-transfer-hook-interface 2.1.0` is only `spl-tlv-account-resolution = "^0.11.0"`, so the pre-fix 0.11.0 release remains a valid dependency resolution under the declared manifests. In `0.11.0`, transfer hook extra account metas could preserve the signer flag for accounts that were not already present in the original transfer CPI account list, allowing a caller-supplied signer extra account to be forwarded into the hook program. Passing unintended signer privileges to an external hook program can create a significant security risk, since the hook program may use that signer authority in its own CPI calls.

`de_escalate_account_meta` in `spl-tlv-account-resolution 0.11.0`



```
46     if let Some((is_signer, is_writable)) = maybe_highest_privileges {
47         if !is_signer && is_signer != account_meta.is_signer {
48             // Existing account is *NOT* a signer already, but the CPI
49             // wants it to be, so de-escalate to not be a signer
50             account_meta.is_signer = false;
51         }
52         if !is_writable && is_writable != account_meta.is_writable {
53             // Existing account is *NOT* writable already, but the CPI
54             // wants it to be, so de-escalate to not be writable
55             account_meta.is_writable = false;
56         }
57     }
```

[tlv-account-resolution/src/state.rs#L46-L57](#)

de_escalate_account_meta in spl-tlv-account-resolution 0.11.1

```
45     if let Some(is_writable) = maybe_highest_privileges {
46         if !is_writable && is_writable != account_meta.is_writable {
47             // Existing account is *NOT* writable already, but the CPI
48             // wants it to be, so de-escalate to not be writable
49             account_meta.is_writable = false;
50         }
51     }
52
53     // Always mark an account as a non-signer
54     account_meta.is_signer = false;
```

[tlv-account-resolution/src/state.rs#L45-L54](#)

Therefore, the reviewed build is safe because the lockfile happens to select `0.11.1`, but the safety boundary is not enforced by the manifests themselves. If the lockfile is deleted, manually changed, regenerated under a stale mirror or vendor source, built with minimal versions, or otherwise resolved differently, `0.11.0` can still satisfy the dependency constraints and reintroduce the unsafe signer forwarding behavior.

It is recommended to declare an explicit direct dependency on `spl-tlv-account-resolution` with a minimum safe version, such as `^0.11.1`. If exact build reproducibility is preferred, the dependency can be pinned to `=0.11.1` instead. This makes the transfer hook signer de-escalation requirement part of DBC's manifest-level dependency constraints instead of relying only on the transitive version selected in Cargo.lock.



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

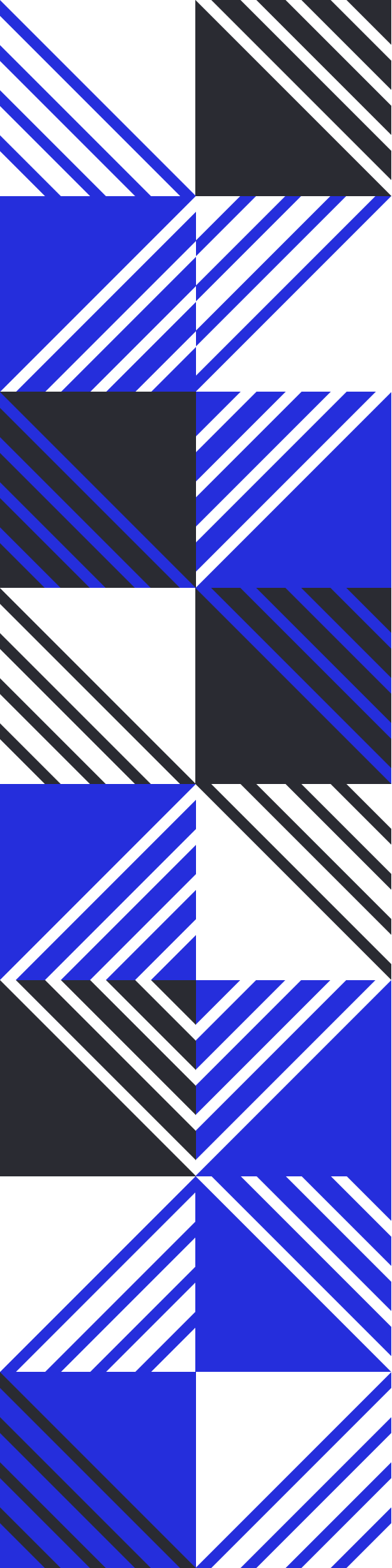
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs